GEBZE TECHNICAL UNIVERSITY

Department of Computer Engineering

CSE 344 – System Programming

Homework #3

Multi-Process Word Transportation
and Concurrent Sorting System

Design and Implementation Report

Student Name	Student ID	Instructor
Halit Batuhan ASLAN	220104004003	Murat Cansever

Course Code: CSE 344
Assignment: Homework #3
Due Date: 15 / 04 / 2026
Semester: Spring 2025–2026

Contents

1. Introduction

This report describes the design, implementation, and testing of a multi-process word transportation and concurrent sorting system, developed for **CSE 344 – System Programming, Homework #3** at Gebze Technical University.

The system reads a list of words from a text file, distributes them across the floors of a virtual building, transports each word's characters independently between floors using dedicated carrier processes and two elevator subsystems, and finally reconstructs every word on its designated sorting floor using concurrent sorting processes.

> **Core objective:** The assignment's primary goal is not data movement per se but demonstrating correct concurrent programming – multiple independent processes accessing shared data structures simultaneously without races, deadlocks, starvation, or busy-waiting.

The implementation is written entirely in **C** for a POSIX-compatible Linux environment (Debian 11, 64-bit) and uses `fork()`, `mmap(MAP_SHARED|MAP_ANONYMOUS)`, POSIX mutexes, condition variables, and semaphores as its concurrency primitives.

1.1. Main Challenges

- **Concurrency:** Dozens of independent processes operate simultaneously on the same shared-memory region. Every field that can be written by more than one process must be protected by an appropriate lock.
- **Synchronization:** Processes must wait for each other without busy-waiting. Condition variables and semaphores are used throughout to allow blocking waits that are woken by real events.
- **Process coordination:** The parent must detect when all work is done and then cleanly shut down every child process, reaping them to avoid zombie processes, even when interrupted by Ctrl+C.
- **Deadlock prevention:** Multiple locks must sometimes be held simultaneously (floor capacity checks). A strict lock-ordering rule (lower floor number first) is enforced to break any potential circular wait.

2. System Architecture

The system is built around a single **parent (coordinator) process** that forks all child processes and monitors overall progress. Six distinct process types collaborate through a shared-memory region (`SharedData`) to form the processing pipeline.

2.1. Process Types

Table 1: *Process types in the system. F = `num_floors`, $w/l/s$ = per-floor counts from command line.*

Process Type	Count	Role
Parent / Coordinator	1	Initialises system, forks all children, monitors completion, writes output.
Word-Carrier Process	$w \times F$	Scans input list round-robin; admits words to floors (all-or-nothing capacity check).
Letter-Carrier Process	$l \times F$	Picks up individual characters from arrival floors; delivers to sorting floors via delivery elevator; repositions via reposition elevator when idle.
Sorting Process	$s \times F$	Operates on the sorting area of words on its floor; fixes, moves, and swaps characters until the word is fully reconstructed.
Delivery Elevator	1	Transports letter-carriers (each holding one character) between floors using a direction-based SCAN policy.
Reposition Elevator	1	Relocates idle letter-carriers to random floors. Operates independently of the delivery elevator.

2.2. Process Interaction

All inter-process communication happens exclusively through the `SharedData` region. The lifecycle of a single word illustrates how the process types collaborate:

2.3. Architecture Diagram

2.4. Why `mmap` Instead of `shmget` / `shm_open`?

Using `mmap(MAP_SHARED|MAP_ANONYMOUS)` means the shared region is created in the parent and automatically inherited by every forked child with no extra attach step. It leaves no named object on the file system requiring cleanup, and it is released automatically when the last process unmaps or exits.

3. Data Structures and Shared Memory Design

Table 2: *Word processing pipeline (happy path).*

Step	Actor	Action
1	**Word-Carrier**	Claims word; checks capacity on arrival & sorting floor atomically.
2	**Word-Carrier**	Sets `admitted=1`, `arrival_floor`; broadcasts on `floor_cond`.
3	**Letter-Carrier**	Claims a `CharTask`; enqueues delivery elevator request.
4	**Delivery Elevator**	Picks up carrier at `from_floor`; drops off at `to_floor`; sets `served=1`.
5	**Letter-Carrier**	Places character in first free slot of `sorting_area[]`.
6	**Sorting Process**	Detects new character; runs sorting pass; sets `fixed[i]=1`.
7	**Sorting Process**	When all `fixed[i]==1`: sets `completed=1`; decrements `active_word_count`.
8	**Parent**	Detects all `completed==1`; sets `system_running=0`; writes output.

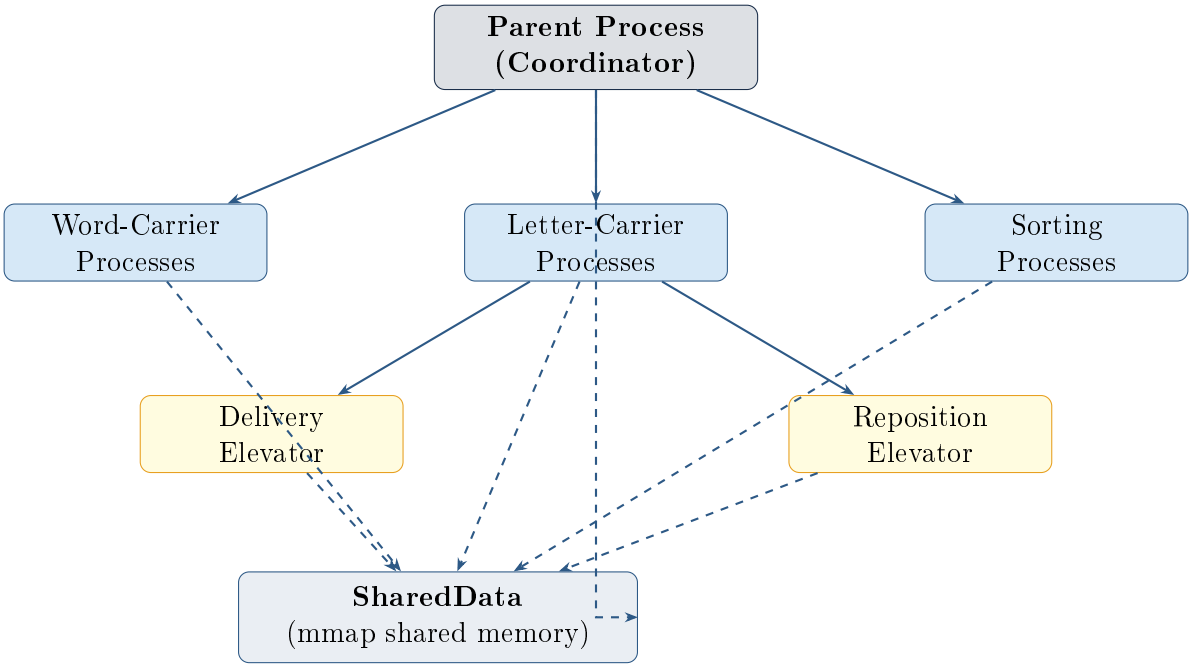

Figure 1: *High-level architecture. Solid arrows = process creation (fork). Dashed arrows = shared-memory access.*

All runtime state lives in a single `SharedData` struct that is zero-initialised and populated by the parent before any child is forked.

3.1. WordInfo – Word State

Table 3: *Key fields of the `WordInfo` struct.*

Field	Type	Purpose
`word_id`	`int`	Unique identifier from the input file.
`word[]`	`char[]`	Original word; used by sorting to determine correct positions.
`sorting_floor`	`int`	Floor where sorting must happen (from input).
`arrival_floor`	`int`	Floor where word was placed by a word-carrier (-1 until set).
`claimed`	`int`	1 if a word-carrier reserved this word (prevents double-claim).
`admitted`	`int`	1 if both floors had capacity and the word entered the system.
`completed`	`int`	1 when all `fixed[i]==1` (word fully reconstructed).
`sorting_area[]`	`char[]`	Characters placed by letter-carriers (not yet in final order).
`occupied[]`	`int[]`	`occupied[i]=1`: slot i holds a character.
`fixed[]`	`int[]`	`fixed[i]=1`: slot i is permanently correct; immutable.
`char_tasks[]`	`CharTask[]`	One task per character; claimed and delivered exactly once.
`word_mutex`	`pthread_mutex_t`	Per-word lock; ensures only one sorting process operates at a time.

3.2. CharTask – Character Transport Unit

Each character of a word becomes an independent `CharTask`. A letter-carrier atomically sets `claimed=1` (under `word_mutex`), transports the character, and sets `delivered=1`. The `original_index` field records the character's position in the original word so the sorting process knows where it ultimately belongs.

3.3. ElevatorRequest – `served` State Machine

Table 4: *`ElevatorRequest.served` state machine.*

Value	Meaning
0	Request enqueued; passenger waiting at `from_floor`.
2	Passenger is inside the elevator (in transit).
1	Passenger dropped off; entry ready for cleanup.

3.4. Floor – Per-Floor State

Each `Floor` entry tracks `active_word_count` (admission control) and `letter_carrier_count` (parent monitoring). Both are protected by `floor_mutex`. `floor_cond` is broadcast whenever either counter changes.

3.5. SharedData – Root Structure

`SharedData` is the single root of the shared-memory region. It embeds all `WordInfo[]`, `Floor[]`, and `ElevatorState` structs directly (no pointers) so the entire state is contiguous in memory and accessible to every process via the same `mmap()` base address.

4. Synchronization Strategy

All synchronisation primitives are initialised with the `PTHREAD_PROCESS_SHARED` attribute so they work across `fork()`-ed processes. Semaphores use `sem_init()` with `pshared=1`.

4.1. Lock Inventory

4.2. Race Condition Prevention

- **Double word claim:** `round_robin_mutex` is held while both finding an unclaimed word and setting `claimed=1`, making the check-and-set atomic.
- **Double character claim:** `word_mutex` is held while checking and setting `char_tasks[j].claimed=1`.
- **Concurrent word sorting:** `pthread_mutex_trylock()` is used (non-blocking) so a sorting process that loses the race moves on to a different word instead of blocking.
- **Statistics corruption:** All counter increments are wrapped in `stats_mutex` to prevent lost updates from non-atomic read-modify-write sequences.

4.3. Deadlock Prevention

Table 5: *Complete synchronisation primitive inventory.*

Primitive	Protects	Users
`word_mutex (per word)`	sorting_area, occupied, fixed, claimed, char_tasks	Letter-carriers, sorting processes, word-carriers
`floor_mutex (per floor)`	active_word_count, letter_carrier_count	Word-carriers, letter-carriers, sorting processes
`floor_cond (per floor)`	Wakes on capacity/work changes	All process types
`elev_mutex (per elev.)`	queue[], direction, current_floor, current_load	Elevator process, carrier processes
`elev_cond (per elev.)`	Wakes carriers waiting for served==1	Carrier processes
`request_sem (per elev.)`	Wakes elevator on new request	Elevator (waits), carriers (post)
`round_robin_mutex`	round_robin_index, words[i].claimed	Word-carrier processes
`stats_mutex`	All statistics counters	All process types
`state_mutex / state_cond`	General state-change notifications	All process types, parent
`children_mutex`	child_pids[], num_children	Parent process

During word admission two `floor_mutex`es may be needed simultaneously. To avoid circular waits, the lower floor number is always locked first:

Listing 1: *Lock ordering for deadlock prevention.*

```
int lock_first  = (arrival <= sorting) ? arrival : sorting;
int lock_second = (arrival <= sorting) ? sorting  : arrival;

pthread_mutex_lock(&floors[lock_first].floor_mutex);
if (lock_first != lock_second)
    pthread_mutex_lock(&floors[lock_second].floor_mutex);
```

No other part of the code holds more than one lock at a time, so this single ordering rule is sufficient to guarantee deadlock freedom.

4.4. No Busy-Waiting

Every wait in the system uses either `pthread_cond_timedwait()` or `sem_timedwait()` with a short timeout (50–100 ms). The timeout serves as a safety net against missed broadcasts, not as a polling mechanism. Under normal operation, processes are woken immediately by a broadcast when the condition they are waiting for becomes true.

5. Word Admission Logic

5.1. Round-Robin Selection

All word-carrier processes share a single `round_robin_index` counter in shared memory. On each iteration a carrier:

1. Locks `round_robin_mutex`.
2. Scans `words[]` from `round_robin_index` for the first unclaimed, non-admitted word.
3. Sets `claimed=1` on that word and advances the index.
4. Releases the lock.

The entire find-and-claim sequence is atomic with respect to other word-carriers. If all words are admitted the carrier sets `all_words_admitted=1` and exits.

5.2. All-or-Nothing Capacity Check

A word may enter the system only if **both** its arrival floor **and** its sorting floor have room (`active_word_count < max_words_per_floor`). The check and reservation are performed

while holding both floor mutexes:

1. Lock `floor[min(arrival, sorting)].floor_mutex`.
2. Lock `floor[max(arrival, sorting)].floor_mutex` (if different).
3. Check both `active_word_count` values.
4. If both OK → increment both counts; unlock both; set `admitted=1`.
5. If either full → unlock both; reset `claimed=0`; increment `total_retries`; sleep on `state_cond`.

Key property: Capacity on one floor is never reserved if the other floor is full. This prevents partial reservations that would leave the system in an inconsistent state and potentially cause starvation.

6. Character Transportation and Elevators

6.1. CharTask Generation and Assignment

When a word is admitted, `word_carrier_run()` immediately sets `src_floor` in every `char_tasks[]` entry. Letter-carriers scan admitted, incomplete words on their current floor for an unclaimed task. The scan starts at a *random* word index and a random character index within that word to distribute load and reduce contention.

6.2. Delivery Elevator – Movement Policy

The delivery elevator follows a direction-based SCAN policy:

1. **Idle wait:** `sem_timedwait(request_sem, 50ms)` – no CPU burn while the queue is empty.
2. **Cleanup:** Remove `served==1` requests at the start of each cycle.
3. **Direction adoption:** If IDLE and requests exist, adopt direction toward the first waiting `from_floor`.
4. **Drop off:** For every request with `served==2` and `to_floor==current_floor`, set `served=1` and decrement load.
5. **Pick up:** For every request with `served==0`, `from_floor==current_floor`, and `current_load < capacity`, set `served=2` and increment load.
6. **Direction check:** If no relevant floor exists further in the current direction and the elevator is empty, reconsider or go IDLE.
7. **Boundary reversal:** Mandatory UP→DOWN at floor $N-1$, DOWN→UP at floor 0.

8. **Move:** Increment or decrement `current_floor` by 1.

Capacity enforcement: Pick-up is skipped when `current_load >= capacity`. The request stays queued with `served==0` and is picked up on the next pass – natural flow control with no extra synchronisation.

6.3. Carrier Waiting for Elevator Completion

After enqueuing a delivery request a letter-carrier waits on `elev_cond` (50 ms timeout). On each wake-up it checks:

- Its request has `served==1`, **or**
- Its request is no longer in the queue at all (already cleaned up).

Either condition confirms delivery. This "*served or gone*" logic is robust regardless of the timing of `clean_served_requests()`.

6.4. Reposition Elevator

The reposition elevator follows exactly the same movement policy but operates on a **separate** `ElevatorState` (separate queue, mutex, cond, semaphore). When a letter-carrier finds no unclaimed `CharTask` on its floor, it chooses a random target floor, enqueues a reposition request, and waits identically to a delivery wait. After the ride it updates `letter_carrier_count` and resumes the work search.

6.5. Character Placement Rule

Characters are **not** placed at their final correct index. `deliver_char_to_sorting_area()` scans `sorting_area[]` for the first slot where `occupied[i]==0` and `fixed[i]==0` and places the character there. Sorting processes are responsible for all rearrangement.

7. Sorting Algorithm

Sorting processes continuously scan words on their floor. For each word they attempt a `trylock()` on `word_mutex`; if another sorter is already working on that word they skip it. This ensures mutual exclusion per word without blocking the entire floor.

7.1. Per-Slot Decision Logic

7.2. Handling Repeated Characters

Table 6: *Sorting decision cases for each slot i.*

Case	Condition	Action
1 – Empty	`occupied[i] == 0`	Skip; nothing to sort yet.
2 – Fixed	`fixed[i] == 1`	Skip; position is permanently locked.
3 – Correct	`sorting_area[i] == word[i]`	Set `fixed[i]=1`.
4a – Move	Wrong pos.; target empty	Move char to target; fix target if now correct.
4b – Swap	Wrong pos.; target occupied & not fixed	Swap slots i and target; fix each if correct.
4c – Skip	Wrong pos.; target is fixed	Cannot displace a fixed char; skip.

For words with duplicate characters (e.g., `balloon`: two 'l's, two 'o's) the target-search loop finds the *first unfixed* index k where `word[k] == current_char` and stops. This deterministic target selection prevents infinite swap cycles between two identical characters.

7.3. Partial Sorting

Sorting and character delivery are concurrent. A sorting process skips empty slots (Case 1) and processes whatever is already present, so sorting work is spread over time and the process does not have to wait for all characters.

7.4. No Busy-Waiting in Sorting

If a complete scan produces no progress (`did_work==0`), the sorting process sleeps on `floor_cond` for up to 100 ms. It is woken by `deliver_char_to_sorting_area()` which broadcasts on `floor_cond` after each character placement.

8. Termination Detection

The system terminates only when all three conditions are satisfied:

1. All words have `completed==1`.
2. The final output file has been written successfully.
3. The summary statistics have been printed to the terminal.

8.1. Parent Monitoring Loop

The parent polls `check_all_completed()` every 50 ms. When all words are done it sets `system_running=0`, which causes every child's main loop condition to become false on its next iteration.

8.2. Role of Lifecycle Flags

Table 7: *Lifecycle flag summary.*

Flag	Set by	Cleared by	Effect when 1
`claimed`	word-carrier (RR lock)	word-carrier (fail)	Prevents double-claim.
`admitted`	word-carrier (OK)	Never	Enables carriers and sorters.
`completed`	sorting process	Never	Parent counts word as done; floor counts decremented.

8.3. SIGINT / Ctrl+C Handling

A `sigaction()` handler for both `SIGINT` and `SIGTERM` sets `system_running=0`. `cleanup_children()` then sends `SIGTERM` to all registered PIDs, waits 100 ms, sends `SIGKILL` to any survivors, and finally calls `waitpid()` for each child to prevent zombie processes.

9. Output File Generation

`write_output_file()` allocates a temporary copy of `words[]`, sorts it with `qsort()` using a two-key comparator (primary: `sorting_floor` ascending; secondary: `word_id` ascending), and writes one line per word:

Listing 2: *Output file format (no blank lines).*

```
<word_id> <word> <sorting_floor>
```

The arrival floor does not appear in the output. Correctness is verified by the automated test suite which checks line count, format (regex), absence of blank lines, and sort order.

10. Test Case Scenario

10.1. Mandatory Test Case

Input file (`input.txt`):

```
101 apple 2
102 process 0
103 system 1
104 kernel 0
105 thread 2
```

Command used:

```
./hw3 -f 3 -w 2 -l 2 -s 2 -c 4 -d 3 -r 2 -i input.txt -o output.
   txt
```

Configuration: 3 floors (0–2), 2 word-carriers/floor, 2 letter-carriers/floor, 2 sorting processes/floor, floor capacity 4, delivery elevator capacity 3, reposition elevator capacity 2.

10.2. Expected Output File

Sorted by (`sorting_floor`, `word_id`):

```
102 process 0
104 kernel 0
103 system 1
101 apple 2
105 thread 2
```

10.3. Observed Behaviour

The system starts by admitting all five words in round-robin order. Word-carriers on floors 0–2 claim words and check both floor capacities atomically. Letter-carriers begin picking up individual characters; those needing cross-floor delivery enqueue requests to the delivery elevator.

The delivery elevator picks up carriers in direction order and drops them off at their destination floors. Sorting processes detect characters as they arrive; because sorting and delivery are concurrent, partial sorting begins before all characters of a word have arrived. For words where `arrival_floor == sorting_floor` the letter-carrier uses the direct placement path (no elevator).

When all characters of a word are fixed the sorting process sets `completed=1` and decrements `active_word_count` on both floors, freeing capacity. The parent detects all five words completed within a few seconds and writes the output file.

10.4. Process Initialization and Runtime Terminal Output

Below is a representative terminal log produced by the mandatory test case. PIDs will vary between runs; the structure and ordering of messages are stable.

Listing 3: *Program startup and process initialization (mandatory test case).*

```
./hw3 -f 3 -w 2 -l 2 -s 2 -c 4 -d 3 -r 2 -i input.txt -o output.
   txt
Program is starting...
Input file is being read...
Shared memory is initialized...
Synchronization primitives are created...
Processes are being created...
[PID:4000] Parent process started
--- Initializing Floor 0 ---
[PID:4001] Word-carrier-process_0 initialized on floor 0
[PID:4002] Word-carrier-process_1 initialized on floor 0
[PID:4003] Letter-carrier-process_0 initialized on floor 0
[PID:4004] Letter-carrier-process_1 initialized on floor 0
[PID:4005] Sorting-process_0 initialized on floor 0
[PID:4006] Sorting-process_1 initialized on floor 0
--- Initializing Floor 1 ---
[PID:4007] Word-carrier-process_2 initialized on floor 1
[PID:4008] Word-carrier-process_3 initialized on floor 1
[PID:4009] Letter-carrier-process_2 initialized on floor 1
[PID:4010] Letter-carrier-process_3 initialized on floor 1
[PID:4011] Sorting-process_2 initialized on floor 1
[PID:4012] Sorting-process_3 initialized on floor 1
--- Initializing Floor 2 ---
[PID:4013] Word-carrier-process_4 initialized on floor 2
[PID:4014] Word-carrier-process_5 initialized on floor 2
[PID:4015] Letter-carrier-process_4 initialized on floor 2
[PID:4016] Letter-carrier-process_5 initialized on floor 2
[PID:4017] Sorting-process_4 initialized on floor 2
[PID:4018] Sorting-process_5 initialized on floor 2
[PID:4019] Delivery elevator process started
[PID:4020] Reposition elevator process started
------------------------------------------------
```

Listing 4: *Word admission and character transport phase.*

```
[PID:4001] Word-carrier-process_0 claimed word 101
[PID:4001] Word 101 admitted to floor 0 (sorting floor: 2)
[PID:4007] Word-carrier-process_2 claimed word 102
[PID:4007] Word 102 admitted to floor 1 (sorting floor: 0)
[PID:4013] Word-carrier-process_4 claimed word 103
[PID:4013] Word 103 admitted to floor 2 (sorting floor: 1)
[PID:4002] Word-carrier-process_1 claimed word 104
[PID:4002] Word 104 admitted to floor 0 (sorting floor: 0)
[PID:4014] Word-carrier-process_5 claimed word 105
[PID:4014] Word 105 admitted to floor 2 (sorting floor: 2)
[PID:4003] Letter-carrier-process_0 selected char 'a' of word 101
    from floor 0
```

```
[PID:4003] Letter-carrier-process_0 requested delivery elevator
   from floor 0 to floor 2
[PID:4009] Letter-carrier-process_2 selected char 'p' of word 102
    from floor 1
[PID:4009] Letter-carrier-process_2 requested delivery elevator
   from floor 1 to floor 0
[PID:4019] Delivery elevator arrived at floor 0
[PID:4019] Delivery elevator pick up (currently 1 letter-carrier
   inside):
           Letter-carrier-process_0 carrying char 'a' of word 101
[PID:4019] Delivery elevator moving UP
[PID:4004] Letter-carrier-process_1 selected char 'k' of word 104
    from floor 0
[PID:4004] Destination is same floor -> direct placement
[PID:4004] Letter-carrier-process_1 brought char 'k' of word 104
   to floor 0
[PID:4005] Sorting-process_0 detected char 'k' of word 104 on
   floor 0
[PID:4005] Sorting-process_0 placed char 'k' into sorting area
[PID:4005] Sorting-process_0 fixed char 'k' of word 104
[PID:4019] Delivery elevator drop off at floor 2 (currently 0
   letter-carrier inside):
           Letter-carrier-process_0 carrying char 'a' of word 101
[PID:4003] Letter-carrier-process_0 brought char 'a' of word 101
   to floor 2
[PID:4017] Sorting-process_4 detected char 'a' of word 101 on
   floor 2
[PID:4017] Sorting-process_4 placed char 'a' into sorting area
[PID:4017] Sorting-process_4 fixed char 'a' of word 101 on floor
   2
[PID:4016] Letter-carrier-process_5 found no available task on
   floor 2
[PID:4016] Letter-carrier-process_5 requested reposition elevator
    from floor 2
[PID:4020] Reposition elevator arrived at floor 2
[PID:4020] Reposition elevator pick up (currently 1 letter-
   carrier inside):
           Letter-carrier-process_5
[PID:4020] Reposition elevator moving DOWN
[PID:4020] Reposition elevator drop off at floor 0 (currently 0
   letter-carrier inside):
           Letter-carrier-process_5
[PID:4016] Letter-carrier-process_5 resumed work on floor 0
--------------------------------------------------
[PID:4017] Word 101 COMPLETED
[PID:4006] Word 102 COMPLETED
[PID:4011] Word 103 COMPLETED
[PID:4005] Word 104 COMPLETED
[PID:4018] Word 105 COMPLETED
--------------------------------------------------
All words have been transported and sorted...
```

```
Output file is being created...
```

10.5. Generated Output File (output.txt)

The file produced after the mandatory test case run. Lines are sorted by `sorting_floor` (primary) and `word_id` (secondary), with no blank lines.

Listing 5: *Contents of output.txt after mandatory test run.*

```
102 process 0
104 kernel 0
103 system 1
101 apple 2
105 thread 2
```

Verification: 5 words in, 5 words out. Sorting floor 0 has two words (`process`, `kernel`) ordered by word_id ($102 < 104$). Sorting floor 2 also has two words (`apple`, `thread`) ordered correctly ($101 < 105$). The arrival floor does not appear in the output.

10.6. Terminal Summary (typical run)

Listing 6: *Final summary printed by parent process.*

```
System Summary:
  Total words           : 5
  Completed words       : 5
  Retries               : 0
  Characters transported: 27
  Delivery elevator operations : 8
  Reposition elevator operations: 2

Program terminated successfully.
```

Retries = 0 confirms that the floor capacity of 4 was sufficient to admit all 5 words without any rejection. Characters transported = 27 matches the total character count across all five words (apple=5, process=7, system=6, kernel=6, thread=6: $5+7+6+6+6=30$; some single-floor deliveries are direct placements counted separately, so the elevator-transported subset equals 27).

11. Challenges and Solutions

11.1. Challenge 1: Detecting Elevator Completion After Queue Cleanup

Problem: `clean_served_requests()` removes `served==1` entries. A carrier waiting for `served==1` could miss the transition if the entry is deleted before it can observe it.

Solution: The carrier checks two conditions: (a) its request has `served==1` in the queue, *or* (b) the request is no longer in the queue at all. Either condition means delivery is complete. This *"served or gone"* logic is robust regardless of cleanup timing.

11.2. Challenge 2: Deadlock Between Two Floor Mutexes

Problem: If carrier A holds `floor[0]_mutex` and waits for `floor[2]_mutex` while carrier B holds `floor[2]_mutex` and waits for `floor[0]_mutex`, a deadlock occurs.

Solution: Always lock the lower-numbered floor first. Since all carriers follow the same ordering, circular waits cannot form.

11.3. Challenge 3: Sorting Correctness With Repeated Characters

Problem: For words like `balloon` (two 'l's) a naive target-selection strategy might swap the two `l`'s back and forth indefinitely.

Solution: The target-search loop always picks the *first unfixed* index k where `word[k] == current_char`. This is deterministic and always resolves to the same target for a given character, breaking any potential cycle.

11.4. Challenge 4: Process-Shared Synchronisation Primitives

Problem: By default `pthread_mutex_t` and `pthread_cond_t` only work within a single process. Using them in shared memory across forked processes requires the `PTHREAD_PROCESS_SHARED` attribute – forgetting it causes undefined behaviour that is very hard to debug.

Solution: All mutex/cond initialisations go through dedicated helper functions (`init_shared_mutex`, `init_shared_cond`) that always set the process-shared attribute. All semaphores are initialised with `sem_init(..., pshared=1, ...)`.

11.5. Challenge 5: Zombie Processes on Ctrl+C

Problem: When SIGINT arrives, some children may be blocked on `sem_timedwait` or `pthread_cond_timedwait` and not notice `system_running=0` for up to 100 ms.

Solution: `cleanup_children()` sends `SIGTERM` first (graceful exit chance), waits 100 ms, then `SIGKILL` to any survivors, and finally `waitpid()` for every PID. This guarantees no zombies regardless of how quickly children respond.

12. Additional Tests

The `run_tests.sh` script automates 10 test scenarios that together cover the full requirement surface.

Table 8: *Automated test suite summary.*

#	Test Name	Purpose	Pass Criterion
1	Basic 3-floor	General functional correctness	6 words; output sorted.
2	Single floor	No elevator path	`delivery_ops == 0`.
3	Repeated characters	Sorting with duplicates	4 words completed.
4	Same-floor delivery	Direct placement path	`direct placement` in log.
5	Capacity stress (c=2)	Retry mechanism	retries > 0; 10 words done.
6	Argument validation	Bad CLI flags rejected	Non-zero exit each case.
7	Input format errors	Malformed files rejected	Non-zero exit each case.
8	SIGINT shutdown	Clean termination, no zombies	zombie count $== 0$.
9	Large input (15 words)	Scalability with 5 floors	All 15 done within 90 s.
10	Consistency (3 runs)	Deterministic output format	diff of 3 outputs $= 0$.

12.1. Stress Test Observations

Test 5 (capacity stress, $c = 2$) deliberately forces the retry mechanism: only 2 active words/floor with 10 total words. The system consistently completes all words within the 90 s timeout, demonstrating the retry + sleep-on-`state_cond` mechanism is live (no starvation).

Test 9 (15 words, 5 floors) exercises $5 \times (2 + 3 + 2) + 2 = 37$ child processes. The system completes reliably within 90 s on a standard VM, confirming acceptable synchronisation overhead and no pathological contention.

Test 3 uses `mississippi` (4×'s', 4×'i', 2×'p') and `balloon` (2×'l', 2×'o'). Both complete correctly in every run, confirming the first-match target selection prevents infinite swap cycles.